

Proyecto Final

Tema: Tres en Raya: Edición Quiz

Estudiante	Carrera Profesional	Asignatura
Estudiante Carmen Aida Valiente Quiroz cvalienteq@ulasalle.edu.pe	Carrera Profesional de Ingeniería de Software	Lenguaje de Programación I

Proyecto	Tema	Duración
Final	Tres en Raya: Edición Quiz	14 horas (2h/día)

Semestre académico	Fecha de inicio	Fecha de entrega
2026 - I	10 Junio 2026	18 Junio 2026

Índice

1. Especificaciones del Proyecto	3
1.1. Objetivos del proyecto	3
1.2. Antecedentes	3
1.3. Equipos, materiales y tecnologías	3
2. Marco Teórico	4
2.1. Programación Orientada a Objetos (POO)	4
2.2. Java Swing	4
2.3. Estructuras de datos	4
2.4. Manejo de eventos	5
3. Diseño del Proyecto	6
3.1. Diagrama de clases	6
3.2. Diagrama de flujo del turno	8
3.3. Reglas del juego	8
4. Implementación	9
4.1. Estructura de archivos del proyecto	9
4.2. Código fuente — Clase <code>Main.java</code>	9
4.3. Código fuente — Clase <code>Jugador.java</code>	10
4.4. Código fuente — Clase <code>Pregunta.java</code>	10
4.5. Código fuente — Clase <code>GestorPreguntas.java</code> (fragmento)	11
4.6. Código fuente — Método <code>verificarGanador()</code> en <code>VentanaJuego</code>	12
4.7. Código fuente — Método <code>actionPerformed()</code> en <code>VentanaJuego</code>	13
5. Compilación y Ejecución	15
5.1. Prerrequisitos	15
5.2. Comandos de compilación y ejecución	15
5.3. Captura de ejecución	15

6. Repositorio GitHub	17
6.1. Estado actual	17
6.2. Archivo <code>.gitignore</code>	17
6.3. Commits planificados	17
7. Pregunta de Analisis	19
7.1. Enunciado	19
7.2. Respuesta	19
7.2.1. Analisis de complejidad Big O	19
7.2.2. Medicion empirica	19
7.2.3. Conclusion	20
8. Entregables	21
8.1. Lista de entregables	21
8.2. Estructura de directorios	21
9. Rubricas	22
9.1. Autoevaluacion	22
10. Referencias	23

1. Especificaciones del Proyecto

1.1. Objetivos del proyecto

- Implementar un juego de Tres en Raya con interfaz gráfica en Java Swing.
- Aplicar los cuatro pilares de la Programación Orientada a Objetos (POO): encapsulamiento, herencia, polimorfismo y abstracción.
- Integrar un sistema de preguntas y respuestas que otorgue puntuación dinámica a cada jugador.
- Demostrar el uso de estructuras de datos (`ArrayList`, matrices bidimensionales) en un contexto real.
- Manejar eventos de interfaz gráfica mediante la implementación de `ActionListener`.

1.2. Antecedentes

Este proyecto tiene como punto de partida una versión previa implementada en **C++**, que utilizaba la misma lógica de negocio: tablero 3x3, verificación de ganador por filas, columnas y diagonales, y sistema de turnos alternados. La decisión de migrar a **Java** se tomó para aprovechar las capacidades gráficas de la biblioteca **Swing** y para aplicar los principios de la POO de manera más natural, dado que Java es un lenguaje orientado a objetos por diseño.

La migración mantuvo la estructura lógica del juego e incorporó:

- Interfaz gráfica con ventana, botones y paneles.
- Sistema de preguntas con banco diferenciado por jugador.
- Puntuación acumulativa y doble condición de victoria.

1.3. Equipos, materiales y tecnologías

- Sistema Operativo Windows 11 Home Single Language: versión 25H2
- Editor de texto Visual Studio Code: versión 1.114.0
- Editor de texto Eclipse Adoptium: versión 17.0.19.10
- Compilador `javac` — Java JDK 21 o superior
- Biblioteca gráfica Java **Swing** (incluida en el JDK)
- Git y cuenta en GitHub asociada al correo institucional

2. Marco Teorico

2.1. Programacion Orientada a Objetos (POO)

La POO es un paradigma de programacion basado en la organizacion del codigo en **objetos**, que combinan datos (atributos) y comportamientos (metodos). Sus cuatro pilares son:

- **Encapsulamiento:** Ocultar los atributos internos de una clase usando modificadores de acceso (**private**), exponiendo solo lo necesario a traves de metodos publicos (**getters/setters**). En este proyecto, la clase **Jugador** declara **nombre**, **simbolo** y **puntos** como privados.
- **Abstraccion:** Modelar entidades del mundo real como clases. **Pregunta**, **Jugador** y **GestorPreguntas** son abstracciones de los elementos del juego.
- **Herencia:** **VentanaJuego** hereda de **JFrame** (con **extends**), obteniendo todos los comportamientos de una ventana grafica sin reimplementarlos.
- **Polimorfismo:** **VentanaJuego** implementa la interfaz **ActionListener**, comprometiendose a definir el metodo **actionPerformed()**, que se comporta diferente segun el boton presionado.

2.2. Java Swing

Swing es la biblioteca estandar de Java para construir interfaces graficas de usuario (GUI). Los componentes principales utilizados en este proyecto son:

Tabla 1: Componentes Swing utilizados

Componente	Uso en el proyecto
JFrame	Ventana principal del juego
JButton	Casillas del tablero (3x3)
JPanel	Paneles de puntajes, tablero e informacion de turno
JLabel	Mostrar nombres, puntajes y turno actual
JOptionPane	Dialogos de bienvenida, preguntas, resultados y revancha
JRadioButton	Opciones de respuesta dentro del dialogo de pregunta
BorderLayout	Distribucion Norte-Centro-Sur de la ventana
GridLayout	Tablero 3x3 y paneles de puntajes en columnas iguales

2.3. Estructuras de datos

- **ArrayList<Pregunta>:** Contenedor dinamico de la biblioteca **java.util**. A diferencia de un arreglo comun, puede crecer en tiempo de ejecucion. Se usa en **GestorPreguntas** para almacenar los bancos de preguntas de cada jugador. Es el equivalente al **vector<T>** de C++.
- **Matriz JButton[3][3]:** Arreglo bidimensional que representa el tablero. Permite acceder a cualquier casilla por su posicion (i, j) en tiempo constante $O(1)$.

2.4. Manejo de eventos

En Swing, los **eventos** son acciones del usuario (clic, teclado). Para responder a un clic en un botón, la clase debe implementar **ActionListener** y sobrescribir **actionPerformed(ActionEvent)**. En **VentanaJuego**, este método identifica el botón clickeado, muestra la pregunta y, si la respuesta es correcta, marca la casilla.

3. Diseño del Proyecto

3.1. Diagrama de clases

El proyecto se compone de cinco clases con responsabilidades claramente diferenciadas, siguiendo el principio de responsabilidad única:

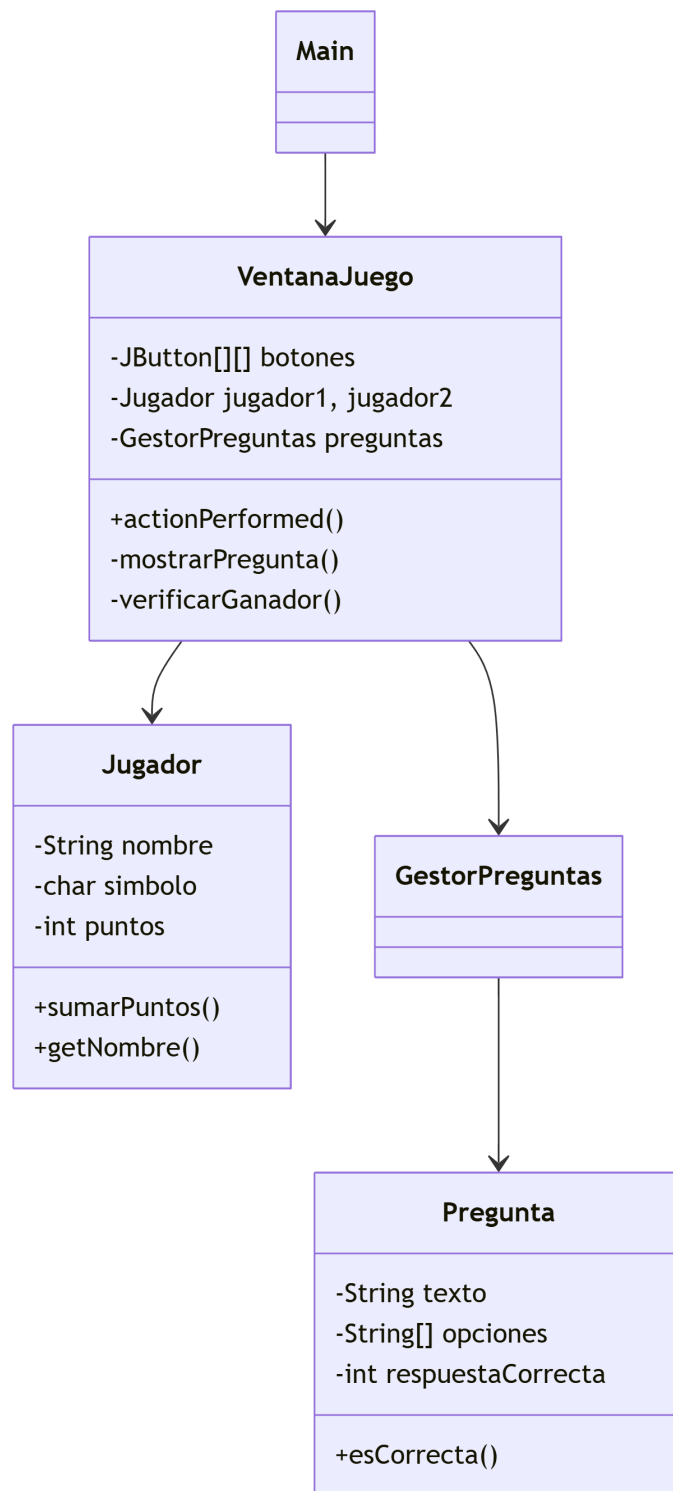


Figura 1: Diagrama de clases — Tres en Raya: Edición Quiz

- **Main:** Punto de entrada. Delega la creación de la ventana al hilo de eventos de Swing mediante `SwingUtilities.invokeLater()`.
- **VentanaJuego:** Controlador y vista principal. Gestiona el estado del juego, la interfaz gráfica y los eventos de los botones.
- **Jugador:** Modelo de datos de cada participante. Encapsula nombre, símbolo (X u O), puntos y color de la interfaz.
- **GestorPreguntas:** Repositorio de preguntas. Administra dos `ArrayList<Pregunta>`, uno por jugador, y gestiona la ronda actual.
- **Pregunta:** Modelo de una pregunta individual con su texto, tres opciones y la respuesta correcta.

3.2. Diagrama de flujo del turno

Cada turno sigue la siguiente lógica de decisión:

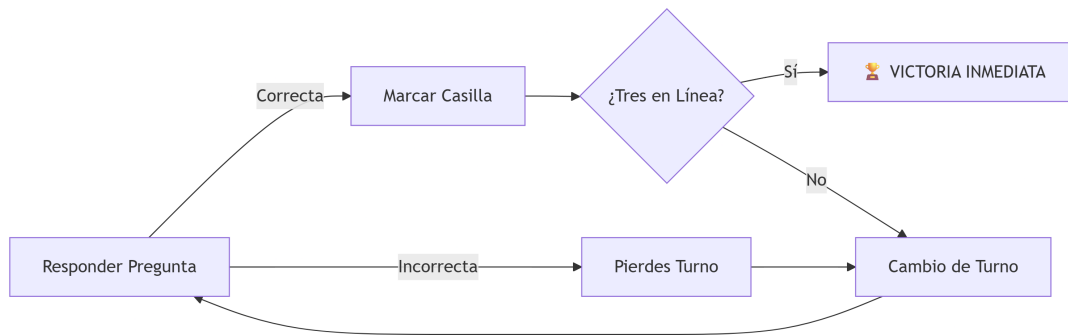


Figura 2: Diagrama de flujo — Lógica de un turno del juego

El jugador selecciona una casilla libre, responde una pregunta de opción múltiple. Si acierta, marca la casilla con su símbolo y se verifica si hay tres en línea. Si falla, pierde el turno sin marcar casilla. En ambos casos, el turno pasa al oponente.

3.3. Reglas del juego

- El tablero tiene 9 casillas numeradas del 1 al 9.
- Para marcar una casilla, el jugador debe responder correctamente una pregunta de opción múltiple.
- Cada respuesta correcta otorga **30 puntos**.
- Una respuesta incorrecta **no marca la casilla** y cede el turno al oponente.
- **Condición de victoria rápida:** El primer jugador en completar tres casillas en línea (horizontal, vertical o diagonal) gana inmediatamente.
- **Condición de victoria por puntos:** Si el tablero se llena sin tres en línea, gana quien acumule más puntos.
- Si los puntos son iguales al llenarse el tablero, el resultado es empate.
- Al finalizar, se ofrece la opción de **revancha** sin cerrar el programa.

4. Implementación

4.1. Estructura de archivos del proyecto

Listing 1: Estructura del proyecto

```
cvalienteq/  
+--- lp1/  
    +--- README.md  
    +--- .gitignore  
    +--- proyecto_final/  
        +--- README.md  
        +--- .gitignore  
        +--- src/  
            |    +--- Main.java  
            |    +--- VentanaJuego.java  
            |    +--- Jugador.java  
            |    +--- Pregunta.java  
            |    +--- GestorPreguntas.java  
        +--- latex/  
            +--- cvalienteq_lp1_26a_proyectofinal.tex  
            +--- cvalienteq_lp1_26a_proyectofinal.pdf
```

4.2. Código fuente — Clase Main.java

Listing 2: Main.java — Punto de entrada del programa

```
1 import javax.swing.SwingUtilities;  
2  
3 public class Main {  
4     public static void main(String[] args) {  
5         // invokeLater pone la creación de la GUI en la cola de eventos de Swing  
6         // Necesario para evitar problemas de concurrencia en la interfaz gráfica  
7         SwingUtilities.invokeLater(new Runnable() {  
8             @Override  
9             public void run() {  
10                 new VentanaJuego().setVisible(true);  
11             }  
12         });  
13     }  
14 }
```

Explicación

- **Línea 7:** `SwingUtilities.invokeLater()` encola la creación de la ventana en el *Event Dispatch Thread* (EDT), el hilo dedicado de Swing para operaciones gráficas. Esto previene condiciones de carrera (*race conditions*).
- **Línea 10:** Se instancia `VentanaJuego` y se hace visible. Todo el control del juego queda en manos de esa clase.

4.3. Código fuente — Clase Jugador.java

Listing 3: Jugador.java — Modelo del jugador (encapsulamiento)

```
1 import java.awt.Color;
2
3 public class Jugador {
4     private String nombre; // Encapsulamiento: atributos privados
5     private char simbolo; // 'X' o 'O'
6     private int puntos;
7     private Color color; // Azul para X, Rojo para O
8
9     public Jugador(String nombre, char simbolo) {
10         this.nombre = nombre;
11         this.simbolo = simbolo;
12         this.puntos = 0;
13         this.color = (simbolo == 'X') ? Color.BLUE : Color.RED;
14     }
15
16     public String getNombre() { return nombre; }
17     public char getSimbolo() { return simbolo; }
18     public int getPuntos() { return puntos; }
19     public Color getColor() { return color; }
20
21     public void sumarPuntos(int puntos) {
22         this.puntos += puntos; // Acumula puntos en el juego
23     }
24
25     public void reiniciarPuntos() { this.puntos = 0; }
26 }
```

Explicación

- **Lineas 4–7:** Todos los atributos son `private`, respetando el principio de encapsulamiento. Ninguna clase externa puede modificar el nombre o los puntos directamente.
- **Línea 13:** Operador ternario para asignar el color según el símbolo: X es azul (Jugador 1), O es rojo (Jugador 2). Esto unifica lógica y presentación visual en la construcción del objeto.
- **Lineas 16–19:** Los *getters* son la única vía de acceso a los datos, garantizando integridad.

4.4. Código fuente — Clase Pregunta.java

Listing 4: Pregunta.java — Modelo de pregunta

```
1 public class Pregunta {
2     private String texto;
3     private String[] opciones; // Array de 3 opciones
4     private int respuestaCorrecta; // Índice 0, 1 o 2
5
6     public Pregunta(String texto, String op1, String op2,
7                     String op3, int respuestaCorrecta) {
8         this.texto = texto;
9         this.opciones = new String[]{op1, op2, op3};
10     }
11 }
```

```
10 // Convertir numero de opcion (1,2,3) a indice (0,1,2)
11 this.respuestaCorrecta = respuestaCorrecta - 1;
12 }
13
14 public String getTexto() { return texto; }
15 public String[] getOpciones() { return opciones; }
16
17 public boolean esCorrecta(int respuesta) {
18     return respuesta == respuestaCorrecta;
19 }
20 }
```

Explicación

- **Línea 11:** La respuesta correcta se recibe del 1 al 3 (más natural para quien crea preguntas) y se convierte internamente a índice 0–2 (necesario para acceder al array). Esta conversión oculta detalles de implementación al exterior.
- **Líneas 17–19:** El método `esCorrecta()` aplica abstracción: quien lo llama no necesita saber como se almacena la respuesta, solo compara.

4.5. Código fuente — Clase `GestorPreguntas.java` (fragmento)

Listing 5: `GestorPreguntas.java` — Banco de preguntas con `ArrayList`

```
1 import java.util.ArrayList;
2
3 public class GestorPreguntas {
4     // ArrayList: contenedor dinámico (equivalente a vector<T> en C++)
5     private ArrayList<Pregunta> preguntasJugador1;
6     private ArrayList<Pregunta> preguntasJugador2;
7     private int rondaActual;
8
9     public GestorPreguntas() {
10         preguntasJugador1 = new ArrayList<Pregunta>();
11         preguntasJugador2 = new ArrayList<Pregunta>();
12         rondaActual = 0;
13
14         // Banco de preguntas - Jugador 1 (X)
15         preguntasJugador1.add(new Pregunta(
16             "Quien es el padre de la P00?",
17             "Alan Turing", "Alan Kay", "Bill Gates", 2));
18
19         preguntasJugador1.add(new Pregunta(
20             "Cuando comenzo la Segunda Guerra Mundial?",
21             "1914", "1939", "1945", 2));
22
23         preguntasJugador1.add(new Pregunta(
24             "Cual es la raiz cuadrada de 81?",
25             "9", "8", "7", 1));
26
27         // Banco de preguntas - Jugador 2 (O)
28         preguntasJugador2.add(new Pregunta(
29             "Quien es el 'Brujo de los Andes'?",
```

```
30         "Miguel Grau", "Andres Avelino Caceres",
31         "Francisco Bolognesi", 2));
32
33     preguntasJugador2.add(new Pregunta(
34         "Formula para el area de un cuadrado?",
35         "Base x Altura / 2", "Lado x Lado", "Pi x Radio^2", 2));
36 }
37
38 public Pregunta getPregunta(int jugador, int ronda) {
39     int indice = ronda - 1;
40     if (jugador == 1) {
41         if (indice >= preguntasJugador1.size())
42             indice = preguntasJugador1.size() - 1;
43         return preguntasJugador1.get(indice);
44     } else {
45         if (indice >= preguntasJugador2.size())
46             indice = preguntasJugador2.size() - 1;
47         return preguntasJugador2.get(indice);
48     }
49 }
50
51 public void siguienteRonda() { rondaActual++; }
52 public void reiniciarRondas() { rondaActual = 0; }
53 public int getRondaActual() { return rondaActual + 1; }
54 }
```

Explicacion

- **Lineas 4–5:** `ArrayList<Pregunta>` usa *generics* para indicar que solo acepta objetos `Pregunta`. Esto da seguridad de tipos en tiempo de compilación.
- **Línea 42:** `get(indice)` accede al elemento en la posición dada en $O(1)$, igual que un arreglo. La diferencia es que `ArrayList` puede crecer dinámicamente con `add()`.
- **Lineas 40–41:** Si se agotaron las preguntas disponibles, se usa la última para no lanzar una excepción `IndexOutOfBoundsException`. Esto hace el juego robusto ante partidas largas.

4.6. Código fuente — Método `verificarGanador()` en `VentanaJuego`

Listing 6: `VentanaJuego.java` — Verificación de ganador en $O(1)$

```
1 private boolean verificarGanador() {
2     char simbolo = jugadorActual.getSimbolo();
3
4     String[] [] valores = new String[3][3];
5     for (int i = 0; i < 3; i++)
6         for (int j = 0; j < 3; j++)
7             valores[i][j] = botonesTablero[i][j].getText();
8
9     // Verificar filas
10    for (int i = 0; i < 3; i++)
11        if (valores[i][0].equals(String.valueOf(simbolo)) &&
12            valores[i][1].equals(String.valueOf(simbolo)) &&
13            valores[i][2].equals(String.valueOf(simbolo)))
```

```
14         return true;
15
16     // Verificar columnas
17     for (int j = 0; j < 3; j++)
18         if (valores[0][j].equals(String.valueOf(simbolo)) &&
19             valores[1][j].equals(String.valueOf(simbolo)) &&
20             valores[2][j].equals(String.valueOf(simbolo)))
21             return true;
22
23     // Verificar diagonal principal
24     if (valores[0][0].equals(String.valueOf(simbolo)) &&
25         valores[1][1].equals(String.valueOf(simbolo)) &&
26         valores[2][2].equals(String.valueOf(simbolo)))
27         return true;
28
29     // Verificar diagonal secundaria
30     if (valores[0][2].equals(String.valueOf(simbolo)) &&
31         valores[1][1].equals(String.valueOf(simbolo)) &&
32         valores[2][0].equals(String.valueOf(simbolo)))
33         return true;
34
35     return false;
36 }
```

Explicación

- **Lineas 4–7:** Se crea una matriz auxiliar `valores[3][3]` copiando el texto de cada botón del tablero. Esto separa la lógica de verificación de los componentes gráficos.
- **Lineas 10–21:** Se recorren las 3 filas y las 3 columnas buscando tres símbolos idénticos. Son 6 comparaciones fijas, independientes del estado del tablero: complejidad $O(1)$.
- **Lineas 23–33:** Se verifican las dos diagonales. El tablero de Tres en Raya tiene exactamente 8 posibles combinaciones ganadoras (3 filas + 3 columnas + 2 diagonales), todas evaluadas aquí.

4.7. Código fuente — Método `actionPerformed()` en `VentanaJuego`

Listing 7: `VentanaJuego.java` — Manejo del evento clic

```
1 @Override
2 public void actionPerformed(ActionEvent e) {
3     if (juegoTerminado) return; // Ignorar clics si el juego termino
4
5     JButton botonClickeado = (JButton) e.getSource();
6
7     // Verificar que la casilla no este ocupada
8     String texto = botonClickeado.getText();
9     if (texto.equals("X") || texto.equals("O")) {
10         JOptionPane.showMessageDialog(this,
11             "Esa casilla ya esta ocupada!");
12         return;
13     }
14
15     // Mostrar pregunta y evaluar respuesta
```

```
16     boolean acierto = mostrarPregunta();
17
18     if (acierto) {
19         // Marcar casilla: buscar posición del botón en la matriz
20         for (int i = 0; i < 3; i++)
21             for (int j = 0; j < 3; j++)
22                 if (botonesTablero[i][j] == botonClickeado) {
23                     marcarCasilla(i, j);
24                     return;
25                 }
26     } else {
27         // Respuesta incorrecta: ceder turno sin marcar
28         cambiarTurno();
29         if (jugadorActual == jugador1) {
30             rondaActual++;
31             gestorPreguntas.siguienteRonda();
32         }
33     }
34 }
```

Explicación

- **Línea 5:** `e.getSource()` retorna el objeto que disparó el evento. Se castea a `JButton` porque sabemos que solo los botones del tablero tienen este listener.
- **Líneas 9–12:** Si la casilla ya muestra X u O, se ignora el clic con un mensaje informativo. Esto evita sobrescribir jugadas previas.
- **Líneas 18–23:** Se recorre la matriz para encontrar las coordenadas (i, j) del botón clickeado y se llama a `marcarCasilla(i, j)`, que actualiza la gráfica y verifica el ganador.
- **Líneas 25–30:** Si la respuesta fue incorrecta, el turno cambia sin marcar casilla. La ronda avanza cuando el Jugador 2 completa su turno (cuando vuelve a ser turno del Jugador 1).

5. Compilación y Ejecución

5.1. Prerequisitos

- Java JDK 17 o superior instalado y configurado en la variable de entorno PATH.
- Verificar instalación con: `java --version` y `javac --version`.

5.2. Comandos de compilación y ejecución

Listing 8: Compilación con soporte UTF-8 y ejecución

```
1 $ cd proyecto_final/src/  
2  
3 $ javac -encoding UTF-8 *.java  
4  
5 $ java Main
```

La bandera `-encoding UTF-8` es indispensable porque el código fuente contiene caracteres especiales del español (tildes, enies). Sin ella, `javac` puede interpretar mal los caracteres y generar errores o texto corrupto en la interfaz.

5.3. Captura de ejecución

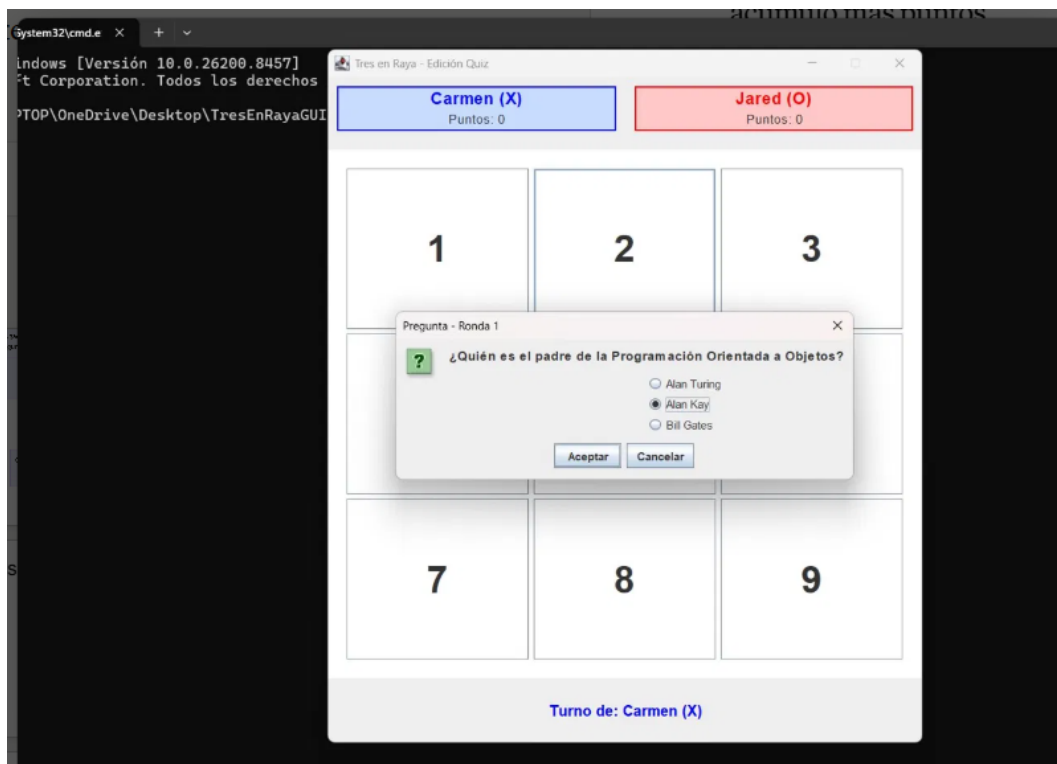


Figura 3: Ejecución del juego: tablero en partida con diálogo de pregunta activo

En la figura se puede observar:

- **Panel superior:** Muestra los nombres ingresados (*Carmen* con símbolo X en azul, *Jared* con símbolo O en rojo) y sus puntajes actuales.
- **Tablero 3x3:** Nueve botones con los números del 1 al 9 representando las casillas disponibles.
- **Dialogo de pregunta:** Ventana emergente con la pregunta de la ronda 1 para el Jugador 1, con tres opciones de radio button. La opción correcta *Alan Kay* está seleccionada.
- **Panel inferior:** Indica que es el turno de *Carmen* (X).
- **Terminal al fondo:** Muestra el directorio de trabajo y confirma que la compilación y ejecución se realizaron desde la línea de comandos.

6. Repositorio GitHub

6.1. Estado actual

El proyecto está en proceso de preparación para su subida al repositorio institucional. Una vez publicado, estará disponible en la siguiente URL:

https://github.com/cvalienteq-coder/lp1/tree/main/proyecto_final

El profesor Richart Escobedo Quispe (usuario: @rescobedoulasalle) será agregado como colaborador del repositorio privado.

6.2. Archivo .gitignore

Para evitar subir archivos compilados al repositorio, se usa el siguiente .gitignore:

Listing 9: Contenido del .gitignore para el proyecto Java

```
1 # Archivos compilados de Java
2 *.class
3
4 # Archivos de IDE
5 .vscode/
6 *.iml
7 .idea/
8
9 # Archivos temporales del sistema
10 Thumbs.db
11 .DS_Store
```

6.3. Commits planificados

Tabla 2: Lista de commits planificados

#	Mensaje del commit	Descripción
1	Agrega estructura inicial del proyecto	Creación de directorios y .gitignore
2	Implementa clases Jugador y Pregunta	Modelos base con encapsulamiento
3	Implementa GestorPreguntas con ArrayList	Banco de preguntas para ambos jugadores
4	Implementa VentanaJuego con GUI Swing	Interfaz gráfica completa y eventos
5	Agrega Main y prueba compilación completa	Integración final y verificación
6	Agrega informe LaTeX del proyecto final	PDF e imágenes del informe

Listing 10: Comandos Git para subir el proyecto

```
1 $ git clone https://github.com/cvalienteq-coder/lp1.git
2 $ cd lp1/
3 $ mkdir -p proyecto_final/src proyecto_final/latex
4 $ cp *.java proyecto_final/src/
5 $ git add .
6 $ git commit -m "Implementa Tres en Raya Edición Quiz - Proyecto Final"
7 $ git push -u origin main
```

7. Pregunta de Analisis

7.1. Enunciado

¿Como podriamos saber con exactitud cual de todas las propuestas algoritmicas es la mas optima? Explique.

7.2. Respuesta

Para determinar con exactitud que propuesta algoritmica es la mas optima, se deben combinar dos enfoques complementarios: el **analisis teorico de complejidad** y la **medicion empirica del rendimiento**.

7.2.1. Analisis de complejidad Big O

El analisis *Big O* permite comparar algoritmos de forma matematica, independientemente del hardware. Mide como crece el tiempo de ejecucion o el uso de memoria en funcion del tamano de la entrada n :

Tabla 3: Complejidades relevantes en el proyecto

Operacion	Big O	Razon
Verificar ganador	$O(1)$	Siempre evalua exactamente 8 combinaciones fijas
Buscar pregunta en <code>ArrayList</code>	$O(1)$	Acceso por indice directo
Buscar un boton en la matriz	$O(n^2)$	Recorre la matriz 3x3 en el peor caso
Buscar en <code>HashMap</code>	$O(1)$ prom.	Alternativa mas eficiente para busqueda por clave

En este proyecto, el algoritmo `verificarGanador()` es $O(1)$ porque el tablero siempre es 3x3: el numero de comparaciones es constante (8 combinaciones) sin importar cuantas casillas esten ocupadas. Esto lo hace mas eficiente que una solucion generica para un tablero $n \times n$ que seria $O(n)$.

7.2.2. Medicion empirica

El analisis teorico no siempre refleja el rendimiento real. Para medir empiricamente en Java:

Listing 11: Medicion de tiempo de ejecucion en Java

```
1 long inicio = System.nanoTime();
2
3 // Algoritmo a medir
4 verificarGanador();
5
6 long fin = System.nanoTime();
7 System.out.println("Tiempo: " + (fin - inicio) + " ns");
```

Se deben ejecutar multiples iteraciones (al menos 1,000) y promediar los resultados para eliminar la variacion por el *garbage collector* de Java y la compilacion JIT (*Just-In-Time*).

7.2.3. Conclusion

Un algoritmo es más óptimo cuando minimiza tanto su complejidad temporal como espacial y esto se confirma empíricamente. En el contexto de este proyecto, dado que el tablero es de tamaño fijo (3x3), todas las operaciones críticas son $O(1)$, por lo que el cuello de botella no está en los algoritmos sino en el rendimiento del framework gráfico Swing.

8. Entregables

8.1. Lista de entregables

- URL al directorio del proyecto en el repositorio GitHub privado con todo el código fuente Java (sin archivos `.class`, temporales ni binarios).
- El profesor debe ser colaborador del repositorio (Usuario: `@rescobedoulasalle`).
- Este informe elaborado en $\text{\LaTeX}$ en formato PDF.
- Capturas de pantalla de la ejecución del juego.
- Diagramas de clases y de flujo generados a partir del código fuente.
- Estructura de directorios del proyecto hasta el nivel 4.

8.2. Estructura de directorios

Listing 12: Estructura completa de entregables

```
1 cvalienteq/  
2 +--- lp1/  
3     +--- README.md  
4     +--- .gitignore  
5     +--- proyecto_final/  
6         +--- README.md  
7         +--- .gitignore  
8         +--- src/  
9             |   +--- Main.java  
10            |   +--- VentanaJuego.java  
11            |   +--- Jugador.java  
12            |   +--- Pregunta.java  
13            |   +--- GestorPreguntas.java  
14         +--- latex/  
15             +--- img/  
16                 |   +--- img_juego.png  
17                 |   +--- diagrama_clases.png  
18                 |   +--- diagrama_flujo.png  
19             +--- cvalienteq_lp1_26a_proyectofinal.tex  
20             +--- cvalienteq_lp1_26a_proyectofinal.pdf
```

9. Rubricas

9.1. Autoevaluacion

- La alumna debera autocalificarse marcando o dejando en blanco las celdas de la columna **Checklist**.
- Si un alumno supera la fecha de entrega, su calificacion sera sobre la nota minima aprobatoria (maximo 24 horas).
- Tabla de niveles de desempeno:

Tabla 4: Niveles de desempeno

Puntos	Nivel			
	Insatisfactorio 25 %	En Proceso 50 %	Satisfactorio 75 %	Sobresaliente 100 %
2.0	0.5	1.0	1.5	2.0
4.0	1.0	2.0	3.0	4.0

Tabla 5: Rubrica del Proyecto Final

Contenido y demostracion		Puntos	Checklist	Estudiante	Profesor
1. GitHub	Repositorio se pudo clonar y se evidencia la estructura adecuada para revisar los entregables.	4	SI		
2. Commits	Hay porciones de codigo fuente asociadas a los commits planificados con explicaciones detalladas.	4	SI		
3. Ejecucion	Se incluyen comandos de ejecucion y pruebas del ejecutable que permitirian replicar el proyecto.	4	SI		
4. Pregunta	Se responde con completitud a la pregunta formulada en la tarea.	2	SI		
5. Diagrama	Se incluyen diagramas de clases y de flujo generados a partir del codigo fuente.	2	SI		
6. Ortografia	El documento no muestra errores ortograficos.	2			
7. Madurez	El informe muestra una evolucion de la madurez del codigo fuente con explicaciones puntuales y precisas, agregando diagramas generados a partir del codigo fuente y refleja un acabado impecable.	4	SI		
Total		22			

10. Referencias

- Oracle Corporation. *Java Swing Tutorial*. <https://docs.oracle.com/javase/tutorial/uiswing/>
- Oracle Corporation. *Java SE 17 API Documentation*. <https://docs.oracle.com/en/java/javase/17/docs/api/>
- Deitel, P. & Deitel, H. (2020). *Java Como Programar* (11.a ed.). Pearson Educacion.
- Sedgewick, R. & Wayne, K. (2011). *Algorithms* (4.a ed.). Addison-Wesley Professional.